САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бекэнд-разработка

Отчёт

Домашнее задание №3 «Тестирование API средствами Postman»

Выполнил:

Клименко Богдан Владимирович

Группа БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать тестирование API фото-соцсети средствами Postman. Написать тесты внутри Postman и объединить их в один комплексный сценарий по основному бизнес-процессу приложения.

Ход работы

1. Подготовка окружения

Для запуска тестов локально поднят сервер приложения (npm run dev), доступный по адресу http://localhost:3000. В Postman созданы:

- Коллекция **PhotoSocial collection**, которая объединяет 16 запросов, покрывающих основной бизнес-процесс.
- Окружение **PhotoSocial Local**, которое хранит переменные, используемые в запросах и заполняемые автоматически по ходу сценария.

Переменные окружения:

Переменная	**Назначение**
baseUrl	http://localhost:3000/api
token	JWT-токен первого пользователя
userId	ID первого пользователя
postId	ID созданного поста
commentId	ID созданного комментария
secondToken	JWT-токен второго пользователя
otherUserId	ID второго пользователя
notificationId	ID уведомления

2. Сценарий тестирования

Разработан комплексный сценарий, повторяющий основной бизнес-процесс фото-соцсети:

1. **Register** - регистрация первого пользователя
2. **Login** - вход и получение JWT-токена
3. **Upload Avatar** - загрузка аватара (multipart/form-data)
4. **Create Post** - создание поста с фотографией (multipart/form-data)
5. **Like Post** - лайк собственного поста
6. **Add Comment** - добавление комментария к посту
7. **Register Second User** - регистрация второго пользователя

8. **Login Second User** - вход второго пользователя, получение его токена
9. **Like Post by Second User** - лайк поста первого пользователя от лица второго
10. **Get Notifications for First User** - получение уведомлений первым пользователем
11. **Mark Notification Read** - отметка уведомления прочитанным
12. **Follow Second User** - подписка первого пользователя на второго
13. **Get Notifications** - повторная проверка уведомлений
14. **Mark Notification Read** - повторная отметка (опционально)
15. **Delete Comment** - удаление комментария
16. **Delete Post** - удаление поста

3. Тестовые скрипты

Для каждого запроса написаны тесты на вкладке Post-response.
Примеры:

Register:

```
pm.test("Status code is 201", () =>
pm.response.to.have.status(201));
var jsonData = pm.response.json();
pm.environment.set("userId", jsonData.id);
```

Login:

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.environment.set("token", pm.response.json().token);
```

Create Post:

```
pm.test("Status 201", () => pm.response.to.have.status(201));
pm.environment.set("postId", pm.response.json().id);
```

Like Post by Second User:

```
pm.test("Status 201", () => pm.response.to.have.status(201));
pm.test("Liked message", () => pm.response.json().message
=== "Liked");
```

Get Notifications for First User:

```
pm.test("Status 200", () => pm.response.to.have.status(200));
var jsonData = pm.response.json();
pm.test("Notifications is array", () => Array.isArray(jsonData));
if (jsonData.length > 0) {
  pm.environment.set("notificationId", jsonData[0].id);
}
```

Delete Comment / Delete Post:

```
pm.test("Status 200", () => pm.response.to.have.status(200));
```

Тесты проверяют корректность HTTP-статусов, наличие обязательных полей в ответах и автоматически заполняют переменные окружения (token, userId, postId, commentId, secondToken, otherUserId, notificationId) для использования в последующих запросах.

4. Результаты тестирования

Все 16 запросов выполнены успешно. Прикрепляю полные ответы, даже с токенами:

Register:


```
{
    "id": 1,
    "username": "tester_12345",
    "email": "test_12345@example.com"
}
```


Login:


```
{
    "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjEsImlhdCI6MTc4
OTUwMTM3NSwiZXhwIjoxNzkwMTA2MTc1fQ.Eac6SS5zRbcSX5IV-
lUzm8moJYK9LY1b38dpfOMi1LE",
    "user": {
        "id": 1,
        "username": "tester_12345",
        "email": "test_12345@example.com"
    }
}
```


Upload avatar:


```
{

    "avatarPath": "/uploads/avatars/1_1789501385725.JPG.jpg"

}
```


Create Post:


```
{
    "userId": 1,
    "originalPhotoPath":
"/uploads/posts/original_1_1789501437088.jpg",
    "previewPhotoPath":
"/uploads/posts/preview_1_1789501437088.jpg",
    "description": "Тестовый пост из Postman",
    "id": 1,
    "createdAt": "2026-09-15T19:43:57.000Z"
}
```

Like Post:


```
{
    "message": "Liked"
}
```


Add Post:


```
{
    "userId": 1,
    "postId": 1,
    "text": "Тестовый комм",
    "photoPath": "/uploads/comment-
photos/comment_1_1789501447331.jpg.jpg",
    "id": 1,
    "createdAt": "2026-09-15T19:44:07.000Z"
}
```


Register Second User:


```
{
    "id": 2,
    "username": "user2_67890",
    "email": "second_67890@example.com"
}
```


Login Second User:


```
{
    "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjIsImlhdCI6MTc4
OTUwMTQ1NywiZXhwIjoxNzkwMTA2MjU3fQ.oVhbB2kPQJyYdogDFcVMWjmMo3voztR
D7aa_wOha0cY",
    "user": {
        "id": 2,
        "username": "user2_67890",
        "email": "second_67890@example.com"
    }
}
```


Like Post by Second User:


```
{
    "message": "Liked"
}
```


Get Notifications for First User:


```
[
    {
        "id": 1,
        "userId": 1,
        "type": "like",
        "relatedUserId": 2,
        "relatedPostId": 1,
        "isRead": false,
```

```
            "createdAt": "2026-09-15T19:44:22.000Z"
        }
    ]
```


Mark Notification Read:


```
    {
        "message": "Marked as read"
    }
```


Follow Second User:


```
    {
        "message": "Followed"
    }
```


Get Notifications:


```
    [
        {
            "id": 1,
            "userId": 1,
            "type": "like",
            "relatedUserId": 2,
            "relatedPostId": 1,
            "isRead": true,
            "createdAt": "2026-09-15T19:44:22.000Z"
        }
    ]
```


Mark Notification Read:


```
    {
        "message": "Marked as read"
    }
```


Delete Comment:


```
    {
        "message": "Comment deleted"
    }
```


Delete Post:


```
    {

        "message": "Post deleted"
    }
```


5. Проверка негативных сценариев

Дополнительно вручную проверены ошибочные ситуации:

- Повторный лайк → 409 Conflict ("Already liked").


```
{
    "message": "Already liked"
}
```

- Попытка удалить чужой пост → 403 Forbidden ("Not your post").

```
{
    "message": "Not your post"
}
```

- Запрос без токена → 401 Unauthorized ("No token provided").

```
{
    "message": "No token provided"
}
```

- Создание поста без фотографии → 400 Bad Request ("Photo required").

```
{
    "message": "Photo required"
}
```

- Подписка на самого себя → 400 Bad Request ("Cannot follow yourself").

```
{
    "message": "Cannot follow yourself"
}
```

- Обращение к несуществующему ресурсу → Error.

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="utf-8">
    <title>Error</title>
</head>

<body>
    <pre>Cannot GET /api/aboba1337</pre>
</body>

</html>
```

Вывод

В ходе выполнения домашнего задания №3 разработан комплексный сценарий тестирования API фото-соцсети средствами Postman. Коллекция охватывает основной бизнес-процесс приложения: регистрацию двух пользователей, создание поста с фотографией, лайки, комментарии, подписки, уведомления и удаление ресурсов. Все запросы сопровождаются автоматическими тестами, проверяющими HTTP-статусы и структуру ответов, а также сохраняющими ключевые данные (ID, токены) в переменные окружения для использования в последующих шагах. Сценарий успешно пройден полностью, включая проверку уведомлений о лайках между пользователями и корректную работу негативных сценариев. Коллекция может использоваться для регрессионного тестирования при дальнейших изменениях API.